

We Don't Need a Large Language Model for That: When Traditional Machine Learning and Encoders Beat Decoder LLMs in Security Classification

Abstract

Large language models (LLMs) have become an attractive option for cybersecurity classification tasks because they are easy to apply to text-like security data and require little task-specific feature engineering. However, LLM inference can be expensive, general-purpose LLM architectures are not optimized for structured classification, and their effectiveness relative to traditional machine learning remains unclear for well-labeled security datasets.

We compare LLMs and traditional machine learning models on two common cybersecurity classification tasks: phishing email detection and network attack classification. Our evaluation includes standalone zero-shot LLM classifiers, XGBoost, frozen and fine-tuned encoder models including ModernBERT, and two hybrid ML-LLM ensemble designs: a router that invokes an LLM only when the ML model has low confidence, and a meta-learner that trains a second-stage model on both ML and LLM outputs.

Across our evaluated datasets, no standalone decoder LLM or decoder-LLM ensemble meaningfully outperforms the strongest traditional ML baselines. On phishing email detection, zero-shot LLMs perform strongly, but embedding-based ML pipelines and fine-tuned ModernBERT match or exceed them: zero-shot Claude reaches 0.970 AUCPR, while fine-tuned ModernBERT reaches 0.998. On network attack classification, zero-shot LLMs perform at or near the class prior across multiple feature representations, while XGBoost remains strong on named and aggregated tabular features; on aggregated flow statistics, XGBoost reaches 0.845 while zero-shot LLMs sit near the 0.403 prior. Fine-tuning an encoder model recovers useful signal from serialized flow statistics, but it ties rather than beats XGBoost.

The ensemble results reinforce this. On the network tasks, routing uncertain examples to an at-chance LLM reduces AUCPR as more traffic is escalated, and stacking decoder LLM outputs with traditional model outputs leaves flow performance unchanged. The picture is more nuanced where the LLM has signal: routing and stacking can improve weaker models, and strong feature representations can make smaller decoder models competitive with larger frontier LLMs. Still, these findings suggest that practitioners building cybersecurity classification systems with sufficiently large, well-labeled datasets should evaluate traditional ML and encoder-based classification methods before adopting decoder LLMs or hybrid LLM ensembles.

Keywords: LLMs for security, intrusion detection, phishing detection, ensembles, feature representation, fine-tuning, AUCPR.

1. Introduction

Large Language Models (LLMs) are easy to deploy, pre-trained on large amounts of data, and require no task-specific training to produce a first result. Naturally, stakeholders begin to ask: **why spend money training traditional Machine Learning (ML) models when an LLM is already available?** Specifically, in security applications, where missing an incident is costly and false positives drown analysts, a similar question may come

up: if LLMs are so capable, why retain the classical ML detection stack at all? This paper addresses that question empirically with rigorous experiments and LLM-ML synergy solutions. Its short answer has two parts: **an LLM can engage with security data only when the input representation is semantically meaningful, and even then, strong traditional ML and encoder-based models can match or beat it.** More subtle questions arise from that answer, such as: *what about tabular data? how do numerical features affect performance? can embeddings improve results? is an LLM decoder model worth the cost and latency for classification?* We explore all of these questions in this paper.

Traditional ML offers mature, well-understood classifiers, but it has not resolved the base-rate fallacy of intrusion detection, where genuine intrusions are extremely rare relative to benign traffic, and even a detector with a very low false-positive rate will produce far more false alarms than true detections, making its alerts unreliable despite high accuracy. Traditional ML models also require continuous retraining to keep up with an evolving threat landscape, and their results are difficult to interpret. LLMs appear to address some of these issues: they can be prompted zero-shot, fine-tuned on a specific text domain, and designed to justify a decision in natural language. However, they bring their own costs: large fine-tuning data requirements, API latency, high memory usage, cost per inference, and a tendency to behave unpredictably on inputs far from their training distribution.

In this paper, we investigate the performance of LLMs in security application for intrusion detection and compare it to traditional ML models. We find that while LLMs are powerful, they are not always the best choice for classification tasks, and that traditional ML models can still outperform LLMs in certain scenarios. We use two diametrically different dataset groups, one natural language dataset, i.e., phishing emails, and a group of tabular, numerical datasets, i.e., network traffic. The goal is to experiment with a dataset that is intuitive for LLMs and a dataset that is challenging. We explore numerical features and embeddings, as well as conversions of tabular data to natural language, to understand the performance of LLMs in different scenarios. Three of our tabular network datasets are produced from the *same* Mirai-botnet capture (Mirsky et al., 2018) at three aggregation levels, giving a controlled natural experiment: identical traffic and labels, different representations. This design lets us attribute differences in LLM performance to *feature semantics* rather than to the difficulty of the detection task or the lack of natural language descriptions. Furthermore, we explore the synergy between LLMs and traditional ML models, proposing two ensemble architectures that combine the strengths of both approaches: the router (Jacobs et al., 1991; Ong et al., 2024) and the meta learner (Wolpert, 1992).

The main contributions of this work are:

1. **A controlled, multi-representation comparison.** We isolate feature representation as the variable governing LLM classification ability, using one phishing-text dataset (Liu, 2023), three aggregation levels of a single Mirai capture (Mirsky et al., 2018), and CIC-IDS2017 (Sharafaldin et al., 2018).
2. **An evaluation discipline that prevents false positives in the methodology.** Every AUCPR (Davis and Goadrich, 2006) is read against the dataset class prior, and per-packet data is split flow-aware (grouping by a 5-tuple) so that no flow appears in both train and test. This discipline is decisive: under a random per-packet split the

Table 1: Datasets used in the study, spanning one natural-language task and four network-intrusion representations. The three Kitsune-Mirai datasets derive from the same raw packet capture at three aggregation levels. AUCPR must be read against the class prior.

Dataset	Domain	Size (total)	Train / Test	Test prior	Feature type
zefang-liu (Phishing Email)	Phishing	17,537	14,029 / 3,508	37.4%	Natural language (email text)
Kitsune-Mirai AfterImage	Network	764,136	611,308 / 152,828	~84%	115 abstract statistical aggregates
CIC-IDS2017	Network	~2.5M (5K sampled)	4,000 / 1,000	17.2%	78 named flow statistics
Kitsune-Mirai pcap (per-packet)	Network	764,137 packets (5K sampled)	random 4,000 / 1,000; flow-aware 4,709 / 291	~84% (random) / ~42.6% (flow-aware)	12 named per-packet fields
Kitsune-Mirai pcap (flows)	Network	36,611 flows	29,288 / 7,323	40.3%	16 aggregated flow statistics

~0.84 prior alone hands an at-chance LLM an AUCPR of 0.84, so a high prior or a leaky split can each manufacture an illusory “LLMs work” result.

3. **Router and Meta Learner ML-LLM ensembles**, with an explicit characterization of when routing improves cost/quality and when it only hurts, and when stacking adds accuracy versus merely down-weighting the LLM.
4. **Evidence that learned representations, not prompting, are the lever for non-language features**: frozen encoder embeddings recover much of the signal in serialized flow statistics that no zero-shot LLM can read (0.74 from a 0.40 prior), fine-tuning nearly all of it (0.83), yet it ties rather than beats traditional ML XGBoost (0.845).

This paper is organized as follows: Section 2 reviews the datasets, model decisions, and evaluation metrics; Section 3 presents our experimental results; Section 4 draws practical conclusions for feature engineering and model choice; Section 5 reviews related work; we conclude in Section 6.

2. Methodology

Our goal is classification of malicious vs benign signal in security datasets, using LLMs and traditional ML models across a variety of datasets and feature representations. This section describes our philosophy of dataset selection, feature generation, model choice, and metrics.

2.1. Datasets and feature representations

We start with the datasets and their feature representations. We use two types of datasets: one natural language dataset, and a group of tabular datasets. The natural language dataset is a phishing email dataset (Liu, 2023), which contains emails labeled as either phishing or benign. The tabular datasets are derived from a Mirai botnet capture (Mirsky et al., 2018), which contains network traffic labeled as either malicious or benign. We create three different representations of the Mirai dataset: raw packets, flow statistics, and incremental statistics (referred to as AfterImage). We also use the CIC-IDS2017 dataset (Sharafaldin et al., 2018), which contains network flow traffic labeled as either malicious or benign with numerical feature representations of the flows, such as duration, packet length and flags. Table 1 summarizes the datasets used in this study, including their domain, size, train/test split when these datasets are used to train or fine-tune, test prior, and feature type. The prior (class prior or base rate) is the fraction of the test set belonging to the positive (malicious) class. Although the three Kitsune-Mirai representations derive from the same capture, their priors differ because the unit of aggregation changes: Mirai’s high-volume floods generate many packets per flow, so counting individual packets (AfterImage, per-packet) inflates the positive-class share to $\sim 84\%$, while counting aggregated 5-tuple flows collapses that same traffic to 40.3% .

The feature types vary across the datasets. The phishing email dataset uses natural language features (the email text), converted to a numerical format using the general descriptive feature set of Cohen et al. (2018)¹, or to embeddings using frozen ModernBERT (Warner et al., 2024). On the other hand, the network datasets use various forms of tabular data, including abstract statistical aggregates, named flow statistics, and named per-packet fields. The flows are aggregated using the tuple $(src_ip, dst_ip, src_port, dst_port, protocol)$. Statistics are incremental and generated from these flows.

2.2. Models and configurations

Our choice of ML model is XGBoost (Chen and Guestrin, 2016) (hereafter XGB), a characteristic, high performant model that is representative of traditional ML for intrusion detection. Our choices of LLMs aim to cover the spectrum of small/large models, reasoning/non reasoning, encoder vs decoder. Specifically, we distinguish three families: *traditional ML* (XGBoost/XGB), *encoder models* (ModernBERT, frozen or fine-tuned), and *decoder LLMs* (generative models prompted zero-shot); our central negative claims concern the decoders. The list of models used in this study includes:

- **XGBoost (XGB)** (Chen and Guestrin, 2016): a traditional ML model that is trained on the phishing email dataset with numerical, handcrafted features and the network datasets. It is used as a baseline for comparison with the LLMs. It is also the gate model in the Router ensemble, where it makes predictions and if the confidence is low, the input is routed to the LLM for further analysis. In the Meta-Learner ensemble, it serves as a base model whose predictions are combined with the LLM’s outputs by a second-stage model.

1. Code available at: [anonymized for review].

- **XGBoost on frozen ModernBERT**: a traditional ML model that is trained on the phishing email dataset using embeddings generated by a frozen ModernBERT model. This allows us to evaluate the performance of XGBoost when it is given a more powerful feature representation derived from an encoder. Specifically this configuration can be considered a type of an “LLM+ML ensemble” where the encoder is used to generate features for the ML model, but the ML model is still responsible for making the final prediction. In this paper, we consider this to be a case of feature engineering. This configuration also serves as the gate in the Router+BERT ensemble and as a base model in the Meta-Learner. Concretely, the frozen ModernBERT-base encoder (`answerdotai/ModernBERT-base`) is loaded as a plain masked-language-model backbone with *no* classification head and *no* gradient updates: each input string is tokenized (max length 512) and passed through the encoder, and we mean-pool the last hidden state across tokens to obtain a single embedding vector per record. XGBoost is then trained on these frozen embeddings exactly as it would be on hand-crafted features.
- **ModernBERT (fine-tuned)**: an encoder-only model that is fine-tuned end-to-end on the phishing email dataset and the network datasets, with a linear classification head (`AutoModelForSequenceClassification`, 2-way softmax) attached on top of the pooled encoder output and trained with cross-entropy loss. Unlike the frozen configuration above, this model performs the classification itself rather than only supplying features to XGBoost. On phishing, we fully fine-tune all 149M parameters (3 epochs, batch size 16, learning rate 2×10^{-5} , max sequence length 512). On the network flow dataset, which is smaller and class-imbalanced, we instead fine-tune with LoRA (rank $r = 8$, $\alpha = 16$, dropout 0.05, adapting the attention W_{qkv} and W_o projections) and a class-weighted cross-entropy loss (3 epochs, batch size 32, learning rate 2×10^{-4} , max sequence length 128), where the positive-class weight is set to the inverse of the training-set class ratio. This allows us to evaluate the performance of the encoder when it has been specifically trained on the task at hand, and to compare it with the frozen-embedding version above to understand the impact of task-specific fine-tuning (and a classification head) versus using the encoder purely as a fixed feature extractor.
- **Open source LLMs (zero-shot)**: we also evaluate the performance of open-source LLMs in a zero-shot setting on the same datasets. This allows us to compare the performance of different LLM architectures and understand if the results are consistent across different models. We used reasoning LLMs such as GPT-OSS ([OpenAI, 2025](#)) and Mistral-7B-Instruct ([Jiang et al., 2023](#)), which are designed to perform well on problem solving tasks, as well as non-reasoning LLMs such as Gemma3:12B ([Gemma Team, 2025](#)) and Llama3.2 ([Llama Team, AI@Meta, 2024](#)).
- **Llama3.2 (fine-tuned)**: a decoder counterpart to the ModernBERT fine-tuning above. We LoRA-adapt Llama3.2-3B (rank $r = 8$, $\alpha = 16$, attention projections only) with a sequence-classification head, using the same training pipeline and data splits as the corresponding zero-shot and ModernBERT-FT runs.

- **Claude Sonnet 4.6** ([Anthropic, 2026](#)) and **Gemini 3.1 Pro** ([Google DeepMind, 2026](#)): two large frontier models, both trained on internet-scale data with the knowledge that they may have had leakage, especially in the phishing dataset.



Figure 1: ML-LLM ensemble architectures used in this paper: the Router (top) escalates from the ML gate model to the LLM only when confidence is low, prioritizing efficiency; the Meta-Learner (bottom) runs the ML models and the LLM in parallel and combines their outputs with a weighted combiner, prioritizing accuracy.

2.3. ML-LLM Ensemble Architectures

Figure 1 shows the ML-LLM ensemble architectures used in this paper:

- **Router:** The purpose of this architecture is to route inference to one or another model based on confidence. Usually, tier 1 is the ML model that infers a class initially and if its confidence is low, then it routes to an LLM. This architecture prioritizes efficiency.
- **Meta Learner:** In this case, both models make predictions independently in parallel. A “Meta-Learner” model (usually Logistic Regression) weighs their outputs to make the final call. This pattern prioritizes accuracy.

2.4. Prompting and leakage control

We purposefully use a simple, zero-shot prompt for all LLMs and datasets, to avoid overfitting the prompt to a particular model or dataset. The prompt is designed to be straightforward and consistent across all evaluations, allowing us to compare the performance of different LLMs on the same footing. The LLMs are expected to analyze the input and provide a prediction along with a confidence score. Both detection tasks share the same prompt template, shown in Figure 2, with only the domain-specific noun, record label, and class names substituted. For temperature, the four open-source decoders (Mistral, Gemma3:12B, Llama3.2, GPT-OSS), served locally via Ollama, are queried at temperature 0.1 to keep their zero-shot verdicts close to deterministic; Claude Sonnet 4.6 is called through the standard Anthropic Messages API with no temperature override, i.e., at the API’s default sampling temperature. We did not tune temperature as a hyperparameter for any model.

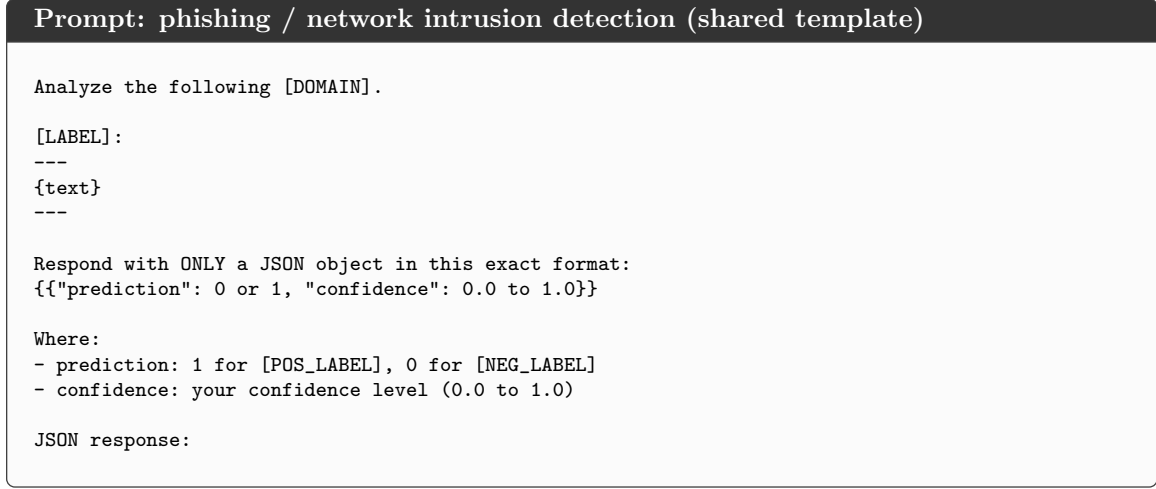


Figure 2: Shared zero-shot prompt template for both detection tasks; only the bracketed slots change by domain. Phishing: [DOMAIN] = “email/message and determine if it is a phishing attempt or legitimate”, [LABEL] = “Message”, [POS_LABEL]/[NEG_LABEL] = “phishing”/“legitimate”. Network intrusion: [DOMAIN] = “network flow record and determine if it represents an attack or benign traffic”, [LABEL] = “Flow”, [POS_LABEL]/[NEG_LABEL] = “attack/malicious”/“benign”. At inference time {text} is replaced by the email body or the serialized flow/packet record, and the model returns a JSON verdict with a confidence used as the ranking score.

For the phishing dataset no feature engineering is applied: the {text} slot is simply the raw email body (likewise truncated to 1000 characters). For the tabular network datasets the LLM never sees a raw numeric vector. Instead, each row is serialized into a short, human-readable record that fills the {text} slot of the prompt, and *every value is labeled with its feature (column) name* so the model knows what each number represents. We adopt two complementary serialization styles: a natural-language sentence (a TabLLM-style serialization (Hegselmann et al., 2023)) for zero-shot prompting, and a compact **key=value** record for the fine-tuned encoder, whose serialization must match its training format exactly, using the natural-language form at inference time instead collapses its AUCPR to chance.

Rather than dumping every column, we curate a small, interpretable subset per dataset so the record stays focused and within the input budget (text is truncated to 1000 characters). For CIC-IDS2017 we include the destination port, flow duration, forward and backward packet and byte counts, inter-arrival mean and standard deviation, mean packet length, and the SYN/ACK/FIN/RST flag counts. For the 115-feature AfterImage representation we keep a fixed positional subset of twelve columns rather than all 115.² For the per-packet and flow representations we include the protocol, source and destination ports, packet length(s), TTL, inter-arrival time, and TCP flags; flag fields are emitted only for TCP records.

Crucially, we enrich every port number with its well-known service name, rendering it as **port(Service)** whenever recognized, ex. 22(SSH), 23(Telnet), 80(HTTP), 443(HTTPS).

2. Per-column semantics for this positional indexing are published at: [TODO: add GitHub link].

This injects domain priors the model already holds from pre-training: Telnet (port 23), for instance, is the primary infection vector of the Mirai botnet, a cue that the bare integer “23” would not convey. One caveat: at the flow level the Mirai-C2 annotation is a near-perfect label proxy (all 13 of 36,611 flows touching port 10240 are attacks), though it touches too few rows ($\sim 0.04\%$) to move the reported AUCPR. Figure 3 shows the same flow rendered in both serialization styles.

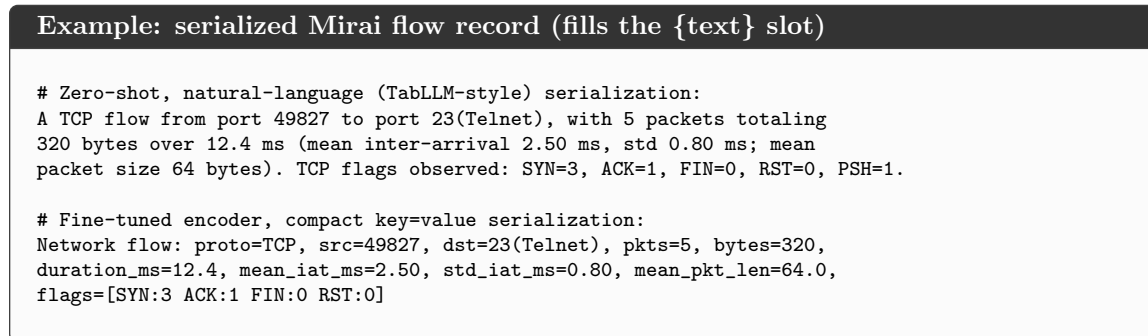


Figure 3: The same Mirai flow serialized for the {text} slot in both styles: a natural-language sentence used for zero-shot prompting, and the compact **key=value** record used to fine-tune the encoder. Feature (column) names label every value, and recognized ports are annotated with their service (here 23(Telnet), the Mirai infection vector).

Text and aggregate datasets use an 80/20 split stratified by label (seed 42). Per-packet data requires more care: thousands of near-identical packets share a 5-tuple, for example a denial of service attack of a scan can have near identical packets that can leak information from training to testing. Therefore, a random per-packet split leaks almost-duplicate rows across train and test. To this end, we split per-packet data with a **flow-aware group/shuffle/split function on the 5-tuple flow_id**, guaranteeing that every packet of a flow lands wholly in train or wholly in test. This is essential: without it, a memorization shortcut inflates the tabular baseline and a high packet-level prior flatters the LLMs, jointly producing a spurious “per-packet LLMs work” result.

2.5. Evaluation metrics

The metrics of choice are AUCPR (Davis and Goadrich, 2006), precision, and recall, standard for evaluating classification on imbalanced datasets. AUCPR is particularly important in security applications with significant class imbalance, providing a more informative measure than accuracy: it captures the trade-off between precision (how many flagged instances are truly positive) and recall (how many true positives are caught) across thresholds, so it directly reflects how well a model identifies the rare positive class.

We interpret AUCPR relative to the class prior, since a random classifier achieves an AUCPR equal to the positive class fraction; this baseline ensures we measure genuine performance rather than being misled by the class imbalance. LLM decisions are converted to a ranking score $P(\text{attack})$ so that AUCPR measures ranking quality rather than thresholded accuracy. To bound API cost, Claude is evaluated on a *proportionally stratified* subsample

that preserves the full-test class prior, keeping its AUCPR baseline directly comparable to the other models'.

All results use a single seed (42); we quantify uncertainty with test-set bootstrap 95% CIs (2,000 resamples, Table 2), and the ML-vs-LLM gaps rest on non-overlapping intervals. Multi-seed replication is future work. In addition to AUCPR, we present selectively the precision and recall to demonstrate that for specific thresholds the LLM predictions show degeneracy and either predict everything as benign (Recall = 0) or everything as malicious (FPR = 1, precision collapsing to the class prior).

3. Results

We present a large spectrum of results that we separated by dataset type, phishing vs network, then we describe fine tuning improvements made to LLMs, and consequently explore the synergy between ML and LLMs with the ensemble architectures router and meta-learner. Finally, we present a data ablation study related to how much data is enough for a traditional ML model, to make the point of cost tradeoffs between ML and LLMs.

3.1. Phishing email detection

Figure 4 shows the results for phishing email detection. Recall that a random, no-skill classifier achieves an expected AUCPR equal to the positive-class prior, which serves as the baseline for evaluating our models.

As expected, the frontier models Claude Sonnet 4.6 and Gemini 3.1 Pro, both trained on the whole internet, will have seen a significant portion of the phishing dataset, therefore their strong performance is not surprising. We thus read the zero-shot decoder phishing numbers as a contamination-inflated upper bound, which only strengthens our thesis: ML and encoder methods still match or beat them. More interesting is that even zero-shot Claude performs strongly (0.970), within 0.03 of the fine-tuned ModernBERT, suggesting it has learned the language patterns associated with phishing emails; zero-shot Gemini follows the same pattern (0.990 standalone, 0.999 behind the embedding gate). ModernBERT FT in turn outperforms XGBoost, capturing nuances the traditional ML model may miss, and XGB with embeddings, also performs well, indicating that LLM-generated embeddings can enhance a traditional ML model. Fine-tuning a decoder shows the same ceiling effect: LoRA-adapted Llama3.2 reaches 0.999 AUCPR standalone, matching fine-tuned ModernBERT.

The router lifts every base model, demonstrating synergy between traditional ML and LLMs. Specifically, Mistral is the lowest performing zero-shot LLM (0.676), but behind the embedding gate it reaches 0.982, nearly matching gated Claude (0.993; note standalone zero-shot Claude is 0.970), a strong result for a much smaller model. Even smaller LLMs can thus be effective when paired with traditional ML and appropriate routing and feature engineering strategies.

In the meta-learner, a second-stage model (logistic regression or a small gradient-boosted tree) is trained on out-of-fold predictions from the base classifiers and every decoder LLM (phishing stacks summarized below; network flow stacks in Table 3), the most generous setting for the LLM: nothing is gated away and the combiner may rely on it as heavily as the data justify.

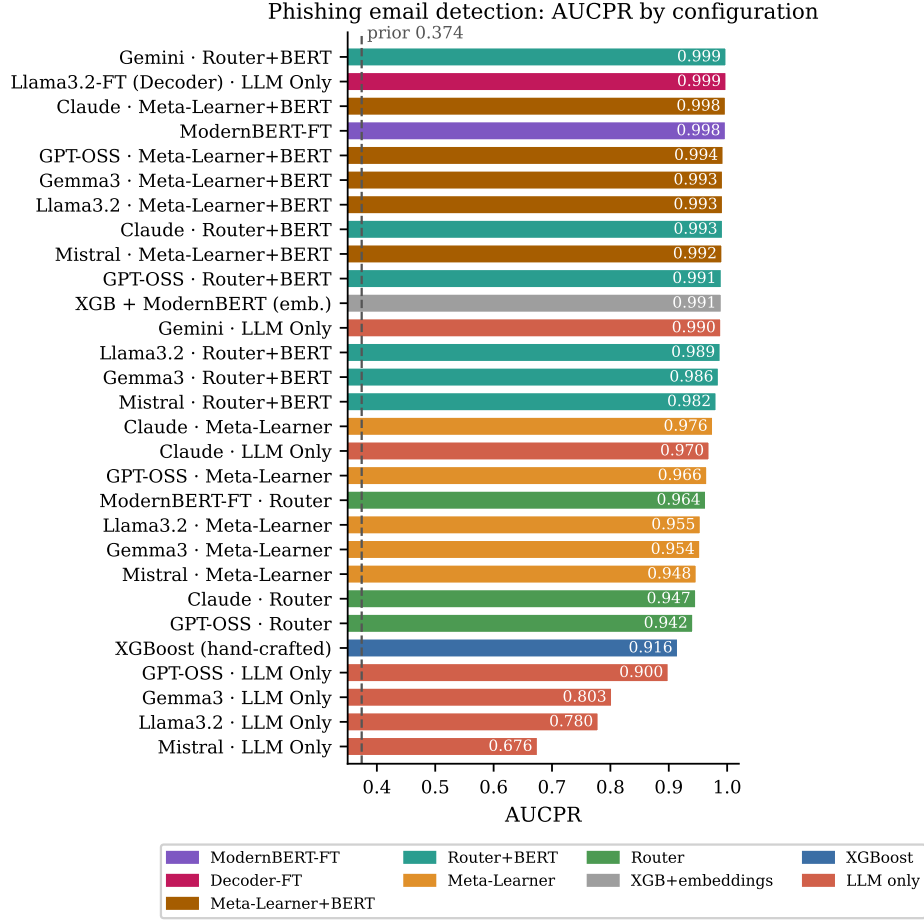


Figure 4: Phishing email detection: AUCPR for every configuration (prior 0.374). Zero-shot Claude (0.970) trails the frozen-embedding pipeline (0.991) and fine-tuned ModernBERT (0.998).

On phishing, stacking is genuinely useful when the base is weak. Combining hand-crafted XGBoost (0.916 AUCPR) with any decoder LLM lifts AUCPR to between 0.95 and 0.98, even though the decoders vary widely on their own, from Mistral (0.676) to GPT-OSS (0.900). Adding frozen ModernBERT embeddings to the stack reaches the ceiling, with the best configuration (XGBoost with BERT embeddings and Claude) attaining 0.998. The lift over the base, however, does not come from the decoder LLM. The same XGBoost-plus-BERT stack with no LLM already scores 0.991, fine-tuned ModernBERT alone scores 0.998 (Section 3.3). The decoder LLM adds at most a few thousandths on top of a strong embedding base, and the best stack (0.998) exactly ties the fine-tuned encoder.

3.2. Network intrusion detection

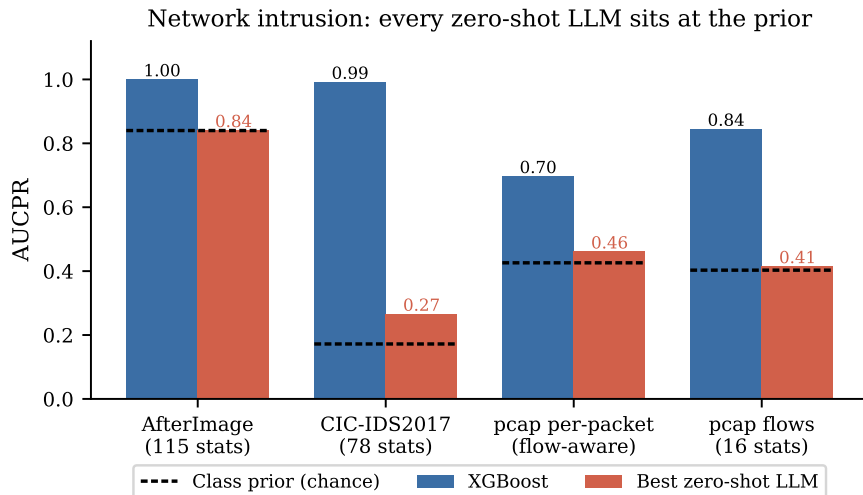


Figure 5: Network intrusion detection: XGBoost vs. the best zero-shot LLM, read against the class prior (dashed). Every zero-shot LLM lands at or near chance; XGBoost is strong except on the leakage-free per-packet split.

Figure 5 shows the results for network intrusion detection. These results tell a different story than phishing: every zero-shot LLM is at or near the prior, i.e., effectively guessing randomly, suggesting that decoder LLMs struggle with tabular feature representations. Reviewing each model at its decision threshold (Table 2) makes the failure concrete: the collapse is not weak ranking but *constant prediction*. On AfterImage (115 abstract statistics, prior 0.84) every zero-shot LLM either predicts benign for all traffic (recall = 0.000, missing every attack) or attack for all of it (FPR = 1.000), while XGBoost’s ranking is near-perfect (1.000 AUCPR). AfterImage’s incremental, decay-windowed features (Mirsky et al., 2018) make a random split a weak leakage guard, so we re-ran XGBoost under a chronological, temporally-ordered per-class split (training on earlier traffic, testing on later) that removes this leakage; its ranking remained near-perfect (AUCPR still ≈ 1.0), confirming the result is not a split artifact, though FPR rises to 0.637 at the default threshold. XGBoost is also trained on more raw columns than the LLMs see (78 or 115 versus the curated dozen). Therefore, we re-trained XGBoost restricted to that same curated subset for both datasets (Table 2, XGBoost (curated)). Note that the AUCPR drops negligibly on CIC-IDS2017 ($0.992 \rightarrow 0.989$, within the full model’s bootstrap CI) and by only 0.004 on AfterImage ($1.000 \rightarrow 0.996$), confirming the gap to any zero-shot LLM is not primarily a removed-information artifact. The pattern repeats on CIC-IDS2017, flows, and per-packet: with one exception, no LLM’s bootstrap AUCPR interval clears the class prior, whereas XGBoost’s lies far above it; the exception, GPT-OSS on CIC-IDS2017 (0.258 [0.213, 0.313], prior 0.172), clears the prior yet remains far below XGBoost (0.992); at the other extreme, Mistral on CIC-IDS2017 (0.142 [0.123, 0.165]) falls significantly *below* the prior, i.e., worse than random guessing. The least-aggregated per-packet representation is the closest the LLMs come, yet even the best of them (Gemma3 0.461) has a 95% confidence interval [0.391, 0.533] that still

Table 2: Operating-point metrics (precision, recall, false-positive rate) at the decision threshold (0.5 for XGBoost; the LLM’s own binary verdict for the LLMs), alongside AUCPR with a bootstrap 95% confidence interval (2,000 resamples). Non-overlapping intervals indicate a real difference, overlapping ones a tie. On phishing the LLMs clear the prior and their intervals separate cleanly; on all four network representations every LLM is at or near chance while XGBoost’s AUCPR dominates, its interval lying entirely above the LLMs’. XGBoost’s operating-point metrics dominate too, except on AfterImage, where catching every attack (recall 1.000) costs a higher FPR (0.637) than a degenerate always-benign LLM achieves trivially. On per-packet the best LLM point estimate (gemma3:12b 0.461) has a CI [0.391, 0.533] that contains the prior 0.426. n is the number of test predictions; Claude is evaluated on a stratified subset to bound cost. For CIC-IDS2017 and AfterImage, XGBoost (curated) re-trains XGBoost on the same curated column subset the LLMs and frozen-BERT embeddings use, rather than all 78 or 115 columns: the AUCPR drop is negligible on CIC-IDS2017 (within the full-feature model’s bootstrap CI) and small on AfterImage (0.004), far short of the gap to any LLM.

Model	n	AUCPR [95% CI]	Recall	Precision	FPR
<i>Phishing email</i> prior = 0.374					
XGBoost	3,508	0.916 [0.905, 0.928]	0.786	0.861	0.076
claude-sonnet-4-6	780	0.970 [0.959, 0.979]	0.799	0.992	0.004
gpt-oss	3,508	0.900 [0.885, 0.915]	0.874	0.798	0.132
llama3.2	3,508	0.780 [0.760, 0.800]	0.803	0.810	0.112
mistral	3,508	0.676 [0.655, 0.697]	0.954	0.680	0.268
<i>AfterImage</i> (115 abstract stats) prior = 0.841					
XGBoost	152,828	1.000 [1.000, 1.000]	1.000	0.892	0.637
XGBoost (curated)	152,828	0.996 [0.996, 0.996]	0.920	1.000	0.002
claude-sonnet-4-6	200	0.840 [0.790, 0.890]	0.000	0.000	0.000
gpt-oss	152,828	0.841 [0.839, 0.843]	0.000	0.000	0.000
llama3.2	152,828	0.841 [0.839, 0.843]	0.000	0.000	0.000
mistral	152,828	0.841 [0.839, 0.843]	1.000	0.841	1.000
<i>CIC-IDS2017</i> (78 named flow stats) prior = 0.172					
XGBoost	1,000	0.992 [0.980, 1.000]	0.988	0.994	0.001
XGBoost (curated)	1,000	0.989 [0.973, 1.000]	0.983	0.994	0.001
claude-sonnet-4-6	353	0.231 [0.167, 0.306]	0.373	0.198	0.303
gpt-oss	1,000	0.258 [0.213, 0.313]	0.459	0.259	0.273
llama3.2	1,000	0.171 [0.147, 0.198]	0.000	0.000	0.000
mistral	1,000	0.142 [0.123, 0.165]	0.000	0.000	0.000
<i>Kitsune flows</i> (16 agg. stats) prior = 0.403					
XGBoost	7,323	0.845 [0.835, 0.855]	0.629	0.781	0.119
claude-sonnet-4-6	419	0.427 [0.370, 0.494]	0.211	0.463	0.184
gpt-oss	7,323	0.403 [0.392, 0.414]	0.000	0.000	0.000
llama3.2	7,323	0.403 [0.392, 0.414]	0.000	0.000	0.000
gemma3:12b	7,323	0.408 [0.396, 0.420]	0.113	0.374	0.128
mistral	7,323	0.394 [0.381, 0.405]	0.003	0.900	0.000
<i>Kitsune per-packet</i> (12 fields, flow-aware) prior = 0.426					
XGBoost	291	0.697 [0.619, 0.764]	0.258	0.889	0.024
claude-sonnet-4-6	246	0.449 [0.376, 0.534]	0.485	0.532	0.308
gpt-oss	291	0.448 [0.381, 0.523]	0.355	0.489	0.275
llama3.2	291	0.416 [0.355, 0.476]	0.202	0.397	0.228
gemma3:12b	291	0.461 [0.391, 0.533]	0.823	0.415	0.862
mistral	291	0.405 [0.345, 0.469]	0.153	0.333	0.228

Table 3: Kitsune-Mirai flows (16 aggregated statistics, prior 0.403; 7,323 test rows): base models and meta-learner stacks. Representative decoders shown are Llama3.2, GPT-OSS, and Claude; the omitted open decoders (Mistral, Gemma3:12B) land at the identical 0.845/0.846. Zero-shot decoder LLMs are at or below the class prior; stacking the open decoders onto a strong base leaves AUCPR essentially unchanged (graceful degradation), whereas Claude’s stacks fall *below* the XGBoost base because Claude was scored on a small cost-capped meta-train subset, so stage-2 has little signal to weight it. Best AUCPR in bold.

Base / stack	LLM	Stage-2	Acc	F1 _{macro}	AUCPR
XGBoost	—	—	0.780	0.762	0.845
XGBoost (frozen BERT)	—	—	0.712	0.685	0.743
LLM only	Llama3.2	—	0.403	0.287	0.403
XGB + LLM	Llama3.2	LR	0.785	0.761	0.845
XGB + LLM	Llama3.2	XGB	0.794	0.761	0.821
XGB + BERT + LLM	Llama3.2	LR	0.787	0.763	0.846
LLM only	GPT-OSS	—	0.403	0.287	0.403
XGB + LLM	GPT-OSS	LR	0.791	0.764	0.845
XGB + LLM	GPT-OSS	XGB	0.794	0.761	0.821
XGB + BERT + LLM	GPT-OSS	LR	0.791	0.764	0.846
LLM only	Claude	—	0.480	0.478	0.434
XGB + LLM	Claude	LR	0.765	0.733	0.810
XGB + LLM	Claude	XGB	0.755	0.707	0.760
XGB + BERT + LLM	Claude	LR	0.760	0.724	0.814

contains the prior 0.426. A second frontier model, zero-shot Gemini 3.1 Pro, follows the same pattern on flows: AUCPR 0.373, below the 0.403 prior.

On flows the LLM in the meta learner architecture contributes nothing measurable. Every zero-shot decoder sits at or below the class prior (between 0.39 and 0.43 AUCPR against a prior of 0.403), and stacking one onto XGBoost with a logistic stage-2 leaves the score at the XGBoost base (0.845; Table 3); the XGBoost stage-2 combiner instead loses a little ground (0.821), consistent with the overfitting noted below. A paired test-set bootstrap with 2,000 resamples of the best LLM stack against the best LLM-free base returns a difference in AUCPR of -0.000 , with a 95% confidence interval of $[-0.0001, 0.0000]$, i.e., an exact tie. The result holds for every decoder we could evaluate at full cache coverage (Llama3.2, GPT-OSS, Mistral, and Gemma3), so it is not a single-model artifact. This contrasts with routing (Figure 7): sending the same at-chance LLM a growing share of traffic does not merely fail to help but pushes AUCPR below XGBoost, whereas the logistic stacking combiner leaves the base unchanged.

The fitted coefficients confirm this is by merit, not mis-specification: the logistic combiner weights the LLM in proportion to its signal, giving it between 22 and 43% of the total on phishing, where the model genuinely reads the email text, but only 1 to 5% on flows, where it is at chance. A logistic second stage also beats a gradient-boosted one on flows, where the nonlinear combiner overfits out-of-fold noise, while the two agree to within about a percentage point on phishing.

Thus, in both domains the meta-learner buys accuracy only where a non-LLM base was weak and a stronger non-LLM base was already available, and the decoder LLM is never the component that reaches the ceiling (practical consequences in Section 4.1). Three caveats: LLM scores are class-conditioned (unaffected by threshold); Claude’s flows stacks fall below the XGBoost base (0.760–0.814 vs. 0.845). Part of the reason is because it is evaluated on a cost-capped subset, unlike the fully covered open decoders. AfterImage and CIC-IDS2017 are omitted from the stacking tables because every configuration there ties the ceiling ($\text{AUCPR} \geq 0.99$, reaching 1.000 on AfterImage), so a base-vs-stack comparison is not meaningful. Per-packet is also omitted, but for a different reason: on the same leakage-free, flow-aware split used in Table 2 (XGBoost base AUCPR 0.697), stacking a decoder LLM onto XGBoost does *not* reach the ceiling either. The best per-packet stack (XGBoost with a Gemma3 LLM feature, logistic stage-2) reaches AUCPR 0.698 – it ties the XGBoost base rather than the ceiling, the same graceful-degradation pattern documented for flows above. An earlier internal per-packet stacking run, used a random per-packet split and its stacks approached $\text{AUCPR} \geq 0.99$. This is exactly the kind of leaky-split artifact that Section 2.4 and the flow-aware per-packet result in Table 2 are designed to rule out. All figures are single-seed, with test-set bootstrap intervals on the stacks.

3.3. Fine-tuning

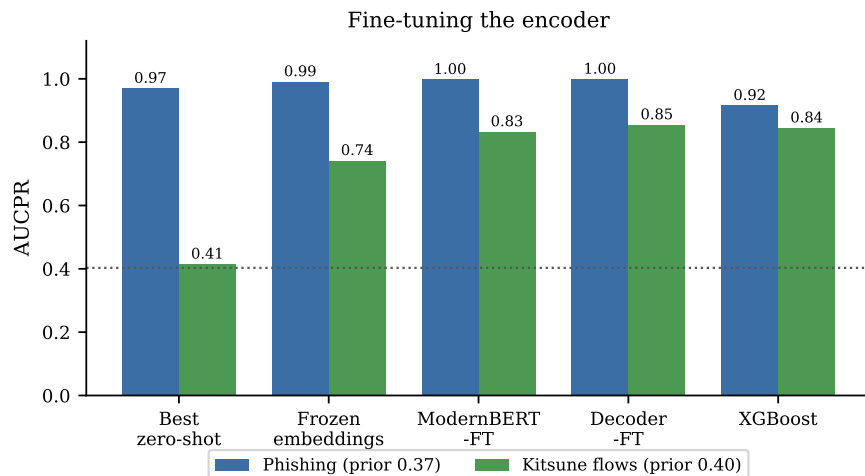


Figure 6: Learned representations, not prompting, unlock serialized flow statistics: on flows, AUCPR rises from chance (zero-shot 0.41) through frozen embeddings (0.74) to encoder fine-tuning (0.83), essentially tying XGBoost (0.84); a fine-tuned decoder goes slightly further (0.86).

We fine-tune an encoder, ModernBERT, and a representative decoder (Llama3.2), on phishing and network datasets to see if learned representations can extract useful information from tabular data, comparing both against their zero-shot versions and XGBoost across all tasks. We use two fine-tuning regimes for the encoder. On phishing we fully fine-tune ModernBERT-base, updating all 149M parameters. On the network datasets, which are

smaller and class-imbalanced, we use parameter-efficient low-rank adaptation (LoRA, rank 8) with a class-weighted cross-entropy loss. The decoder is fine-tuned with LoRA (rank 8) on both tasks. In all cases the fine-tuned model must see the same input format at inference time as during training, i.e., a compact **key=value** record.

Figure 6 compares the phishing and networking best zero-shot LLMs with fine-tuned ModernBERT and the fine-tuned decoder. Fine-tuning moves the needle on the network datasets: on flows, AUCPR rises from chance (zero-shot 0.41) through frozen embeddings (0.74) to encoder fine-tuning (0.83), essentially tying XGBoost (0.84); on phishing all learned methods are near-ceiling, so the gain from fine-tuning is smaller in absolute terms, but it still lifts AUCPR from 0.97 to 0.998, a significant improvement given the already high performance of the zero-shot model. Thus, while zero-shot decoder LLMs struggle with tabular data, a fine-tuned encoder can extract useful information from it.

Two implications follow Figure 6. First, the representation gap on flows is specific to *zero-shot prompting*: the signal is present and learnable, with frozen embeddings recovering much of it and weight updates most of the rest. Second, fine-tuning closes the gap to gradient boosting on flows: the encoder ties it, and, as the decoder result below shows, weight updates can modestly surpass it too.

A LoRA-fine-tuned decoder follows the same trajectory on flows, and slightly further: standalone Llama3.2 reaches 0.855 AUCPR, modestly exceeding both fine-tuned ModernBERT (0.833) and XGBoost (0.845).

3.4. ML-LLM ensembles and the cost/quality trade-off

Figure 7 shows the impact of routing on flow detection. Across multiple thresholds that route higher traffic volumes to the LLM, AUCPR decays monotonically as more traffic is routed to the at-chance LLM, never reaching XGBoost alone (0.845). Routing is thus a cost lever rather than an accuracy lever here: it directs only uncertain cases to the LLM but does not improve overall accuracy over XGBoost alone. The same figure plots the meta-learner stack for reference: instead of decaying, it ties the XGBoost base, because stacking degrades gracefully to the base rate when an input carries no signal, while routing actively trades accuracy away as it leans on that input.

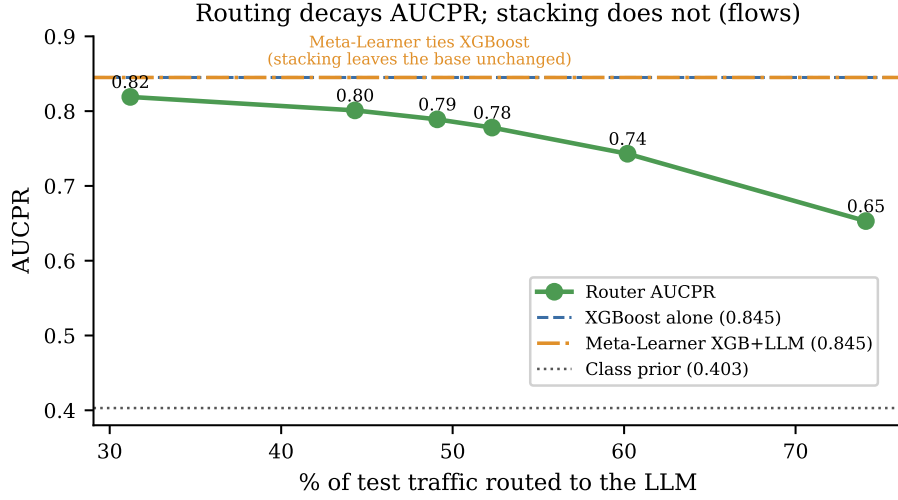


Figure 7: Router confidence-threshold sweep on flows: as more traffic is routed to an at-chance LLM, AUCPR decays monotonically and never reaches XGBoost alone (0.845); the reference meta-learner stack ties the XGBoost base instead of decaying.

Conversely, when the LLM tier has signal, a strong gate makes model choice almost irrelevant. Once a strong embedding gate is used, a small local model (Mistral, 0.982 behind the gate) nearly matches gated Claude (0.993): the AUCPR spread across models collapses from 0.29 as standalone zero-shot classifiers (Claude 0.970 down to Mistral 0.676) to 0.01 once gated. This suggests that when the LLM provides a strong embedding, the choice of the specific LLM becomes less critical, as even a smaller model carries enough signal for the ML model to perform well.

Thus, ensembles are a powerful tool for managing the cost/quality trade-off, but their impact on accuracy depends on the underlying LLM performance and feature quality: when the LLMs are at chance, routing does not improve accuracy; when they provide strong embeddings, even smaller models perform well.

3.5. Data volume ablation

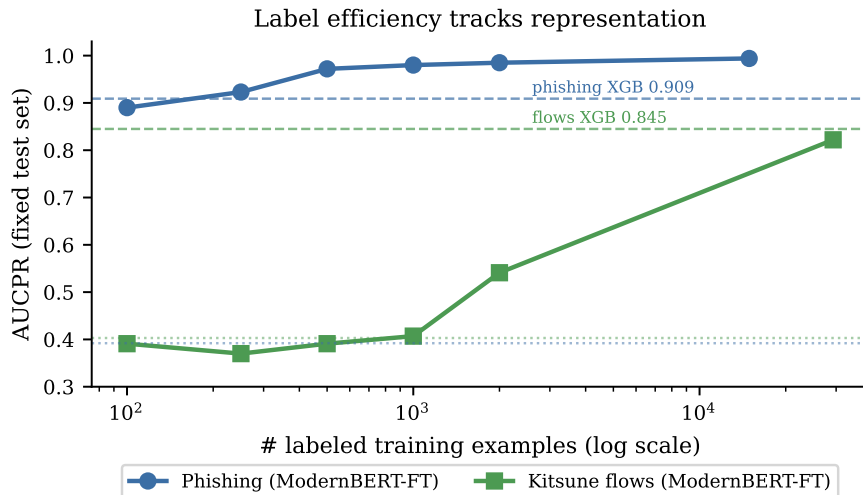


Figure 8: Data-volume ablation (fixed test set): on phishing, fine-tuning beats hand-crafted XGBoost by 250 labels and ties zero-shot Claude by 500; on flows it stays at the prior below 2,000 labels and needs the full set.

A general argument for not using traditional ML is the large amount of data required to train a model. In this section, we demonstrate that this is not necessarily a strong argument. Holding the test set fixed and varying only the number of training labels shows that label efficiency itself tracks the representation (Figure 8). Specifically for phishing detection that is language-friendly, fine-tuning beats hand-crafted XGBoost by 250 labels and ties zero-shot Claude by 500; for flow detection that is language-unfriendly, the LLMs stay at the prior below 2,000 labels and need the full set to tie XGBoost. Label efficiency thus tracks the representation: language-friendly tasks require far fewer labels to reach strong performance than language-unfriendly ones. Even so, 2K labels is not a large amount of data, and the critical jump in performance demonstrates how performant ML models can be with a small amount of data.

4. Discussion

In this section, we address practical research and development questions for using ML and LLMs in security applications.

4.1. Do decoder LLMs earn their place in an ensemble?

On the well-labeled security tasks we study, a text-generating LLM never defines the accuracy frontier, and on cost it is dominated on every architectural axis we can reason about: for every baseline we tried, an LLM-free model, i.e., XGBoost, a frozen-BERT-embedding classifier, or a fine-tuned encoder, matches or beats the LLM-augmented ensemble while being far smaller, locally deployable, and a single forward pass rather than autoregressive generation or a metered API call. The practical implication is that, given a sufficiently large

Table 4: Feature type vs LLM utility.

Feature type	Classical-ML quality	Zero-shot LLM utility	LLM via fine-tuning
Abstract statistics (After-Image)	Very high	None (degenerate)	–
Named flow statistics (CIC-IDS2017)	Very high	None (at chance)	–
Aggregated flow statistics (flows)	High	None (at chance)	$\approx$ XGB (0.833 vs 0.845)
Per-packet named fields (flow-aware)	Moderate (0.697)	None (at chance)	–
Natural language (email text)	Moderate	Strong (0.68–0.97)	Best (0.998)

and well-labeled dataset, the right first move is traditional ML with good feature extraction (including embeddings), not a decoder LLM.

The evidence is sharpest where the LLM should do best. On flows the decoder contributes nothing (paired difference -0.000 , 1 to 5% stage-2 weight; Section 3.2); on phishing, its home turf, the best decoder stack lifts an XGBoost-plus-BERT base from 0.991 to 0.998 (single-seed) but only matches LLM-free options (fine-tuned ModernBERT, 0.998) that are cheaper to serve. The defensible conclusion is domination rather than parity: the LLM can contribute a sliver on text, yet a cheaper non-LLM model always equals or beats it.

These conclusions are scoped to a small set of decently large, well-labeled security datasets and to simple ensembles (confidence routing and out-of-fold stacking). We would expect a decoder LLM to earn its place on tasks that are hard to label or that require multi-step reasoning, for example correlating evidence of a cross-system vulnerability, which is precisely the regime we do not study here. Extending the comparison to messier and lower-signal datasets is left to future work.

4.2. Which features are the best?

Table 4 demonstrates that the results are not as trivial as: "natural language features are best for LLMs, and tabular features are best for traditional ML". Granted, the phishing dataset demonstrates the capability of LLMs to learn the distribution and patterns of language and recognize malicious vs benign emails. However, the network story shows the full spectrum of the LLM and ML capabilities. Even though the afterimage Kitsune dataset has a large variety of features (115 columns), zero-shot LLMs still perform abysmally compared to XGBoost, struggling with aggregated statistics and meta-features that describe time series of network traffic; restricting XGBoost itself to the same curated subset the LLMs see barely moves its AUCPR (Section 3.2). This demonstrates that the gap is not simply a matter of the LLM seeing fewer columns. On the other hand, when the features are more raw and closer to the original data, such as in the per-packet and flow representations, zero-shot decoder LLMs still land at chance, but the signal is learnable: on flows, frozen embeddings and fine-tuning recover most of it.

Table 5: Practical recommendations.

Use case	Recommended approach	Rationale
Phishing email classification	ModernBERT-FT, Router+BERT, or zero-shot Claude	Quality near-equivalent (0.970–0.998); choose on cost/latency
Phishing without ML infrastructure	Zero-shot Claude	0.970 AUCPR, viable standalone
Network intrusion, aggregate features	XGBoost; fine-tuned ModernBERT only with abundant labels	Zero-shot LLMs at chance; FT needs the full label set to tie XGB (0.833)
Network intrusion, scarce labeled data	XGBoost	No FT signal below $\sim 2,000$ labels; XGBoost is reliable under scarcity
Network intrusion, per-packet features	XGBoost with a flow-aware split	Zero-shot LLMs at chance once leakage is removed
Cost-constrained LLM deployment	A small local model behind an embedding gate	Gap collapses to ~ 0.011 once gated; LLM used on only $\sim 4\%$ of inputs
Any new domain	Measure LLM-only AUCPR against the class prior on a leakage-free split first	A high prior or leaky split can manufacture a false “LLMs work” result

4.3. Which models are the best for each use case?

Table 5 cites the recommendations per use case based on our experimentation. Surprisingly, ModernBERT-FT and Router+BERT can be the best tradeoff between speed, cost, and quality for email classification, given the infrastructure and know-how to train them; a frontier model such as Claude is an easy, high-quality solution for language-related security classification. The story is different for tabular data, where specialized models are required for all use cases. Finally, it is worth evaluating the proper tool for any new use case such as alert classification, endpoint logs, and malware.

4.4. Model tradeoffs and cost/quality trade-offs

Table 6 summarizes the cost structure of the evaluated model families. The asymmetry that matters at security volumes is that training cost is paid once while inference cost accrues per record: a metered, autoregressive API call for every flow or email is dominated by a locally served single-pass model of equal accuracy (Warner et al., 2024), and our data ablation (Section 3.5) shows the training side needs only modest labeled data. Small fine-tuned encoders additionally fit edge deployments and keep sensitive records on-premises. Finally, the reasoning decoder (GPT-OSS) and the non-reasoning decoders (e.g., Llama3.2) show only a small gap on these classification tasks; because the models differ in family and size this is an observation rather than a controlled finding, but the non-reasoning models are faster and cheaper to run.

4.5. TabPFN-3 as a tabular base learner

We kept tabular pre-trained transformers such as TabPFN-3 out of the main comparison to avoid diluting the LLM-versus-ML message, but its behavior as a base learner sharpens our thesis rather than complicating it (Table 7). With no task-specific training, TabPFN-3 beats XGBoost on identical features in both domains ($0.916 \rightarrow 0.953$ phishing, $0.845 \rightarrow 0.856$

Table 6: Cost structure of the evaluated model families (architectural comparison; we did not benchmark wall-clock latency or dollar cost). Traditional ML and encoders pay a one-time training cost and classify with a single forward pass per record; decoder LLMs skip training but generate autoregressively for every record, and a frontier API adds metered per-call pricing and sends data off-premises.

Model	Params	Training cost	Per-record inference	Deployment
XGBoost (hand-crafted)	—	one-time fit, CPU	single pass, CPU	local, edge
XGBoost on frozen	149M	one-time fit of head,	one encoder pass +	local
BERT	(frozen)	CPU	head	
ModernBERT fine-tuned	149M	one-time full FT or	single encoder pass	local, edge
		LoRA, one GPU		
Open decoder, zero-shot	1B–20B	none	autoregressive generation	local GPU
Claude Sonnet 4.6	undisclosed	none	autoregressive, metered per call	hosted API

Table 7: TabPFN-3 vs. XGBoost as the tabular base learner (AUCPR), on identical features with no training for TabPFN-3. TabPFN-3 dominates XGBoost on both domains and both feature representations; as the head on frozen ModernBERT embeddings it reaches 0.997 on phishing, within 0.001 of fully fine-tuned ModernBERT. Prior 0.374 (phishing), 0.403 (flows).

Base learner / features	Phishing AUCPR	Flows AUCPR
XGBoost (hand-crafted)	0.916	0.845
TabPFN-3 (hand-crafted)	0.953	0.856
XGBoost (frozen BERT)	0.991	0.743
TabPFN-3 (frozen BERT)	0.997	0.797

flows), and as the head on frozen BERT embeddings it reaches 0.997 on phishing, within 0.001 of fully fine-tuned ModernBERT. It raises the strength of the *base*, not the value of the LLM in the loop: stacking a decoder LLM onto a TabPFN base behaves just as it does onto XGBoost. And because TabPFN-3 is used in context with no training, closer to how we prompt an LLM than to a trained XGBoost, it shows the decoder LLM’s failure is not a lack of task-specific fitting: a no-training *tabular* foundation model still dominates it, so the problem is that the LLM is a text generator.

5. Related Work

Our study sits at the intersection of three lines of research: traditional ML for intrusion detection, LLMs for security classification, and ensembles combining the two.

Traditional ML for intrusion detection. Machine learning for separating malicious from benign signal in security data is mature and well surveyed (Buczak and Guven, 2016). On network traffic, tree-based ensembles, XGBoost (Chen and Guestrin, 2016) foremost, remain the workhorse: they learn sharp decision boundaries from hand-crafted flow and packet statistics and infer cheaply at scale, repeatedly topping intrusion-detection leader-

boards (Farnaaz and Jabbar, 2016). The same holds for phishing, where engineered lexical and host-based URL features feed conventional classifiers to strong effect (Sahingoz et al., 2019). This establishes the strong baseline any LLM detector must beat; its persistent challenges, the base-rate fallacy under heavy class imbalance and the need for continual retraining, are the openings that motivate the turn to LLMs.

LLMs for security classification. A growing body of work prompts or fine-tunes LLMs for security detection, most visibly in the language-friendly phishing domain, where zero-shot and fine-tuned models reach high accuracy on email and URL text (Koide et al., 2024). Tabular network data is harder; the standard remedy serializes each row into a labeled, human-readable record, as in the TabLLM serialization (Hegselmann et al., 2023) we adopt for zero-shot prompting, while encoder fine-tuning with LoRA (Hu et al., 2022) on bidirectional backbones such as ModernBERT (Warner et al., 2024) is the parallel route for non-language features. Our results temper this optimism: once AUCPR (Davis and Goadrich, 2006) is read against the class prior and per-packet data is split flow-aware, the surviving LLM signal is confined to genuinely semantic inputs, and on serialized flow statistics learned representations, not zero-shot prompting, are the lever: fine-tuning an encoder ties XGBoost, and a fine-tuned decoder narrowly surpasses it (Section 3.3).

Ensembles of LLMs and ML. Stacking (Wolpert, 1992) and mixture-of-experts routing (Jacobs et al., 1991) are the classical templates, and both reappear in the LLM era: EnStack stacks code-understanding LLMs under meta-classifiers including XGBoost for vulnerability detection (Ridoy et al., 2024), while RouteLLM (Ong et al., 2024) and the cost-driven cascades of FrugalGPT (Chen et al., 2023) escalate only hard queries to an expensive model. These cost/quality framings inform the Router and Meta Learner ensembles we evaluate. Our contribution is to characterize, in the security setting, when this combination pays off: the question is not whether to combine an LLM with traditional ML, but which model to place behind the gate so the combination is both accurate and cheap to serve.

6. Conclusions and Future Work

This paper asked whether LLMs should replace, or at least routinely supplement, traditional ML models for security classification. On the tasks we studied, the answer is mostly no. For well-labeled phishing and network intrusion datasets, strong traditional ML and encoder-based methods were enough to match or beat standalone decoder LLMs and ML-LLM ensembles.

The network results make this clearest. Across the AfterImage, CIC-IDS2017, per-packet, and flow representations, zero-shot decoder LLMs stayed near the class prior while XGBoost performed well on the tabular features. Routing to the LLM only lowered AUCPR as more traffic was escalated; stacking was safer but still did not beat the traditional model; and fine-tuning tied XGBoost on serialized flow statistics for the encoder, and narrowly beat it for the decoder. Phishing was the LLMs’ best case, since the input is natural language: the decoder LLMs did far better than on the network data and could lift weak baselines. Yet they still did not define the best classifier, because fine-tuned ModernBERT and embedding-based ML pipelines matched or beat them. Language-model representations were useful, but text-generating LLM inference was not the critical ingredient.

Even so, the ensembles were not without value: they did not raise the ceiling but they raised the floor. Behind a strong embedding gate, a small, inexpensive model recovered most of a frontier LLM’s quality while that costly model was invoked on only a small share of inputs. Ensembles may also help where traditional ML is weakest, on the unknowns it was never trained to recognize, such as novel attacks or shifting threat distributions; we leave a careful test of this conjecture to future work.

These conclusions are limited to a small set of well-labeled cybersecurity classification tasks. They may not hold for tasks with very few labels, noisy labels, ambiguous ground truth, or no fixed dataset at all. They also may not hold for tasks that require multi-step reasoning rather than single-record classification. Future work should test LLM agents that can call classifiers, inspect evidence, and combine results across systems, for example in vulnerability discovery or codebase-level security analysis. Those settings may justify the extra cost and latency of LLM inference. For the classification settings studied here, traditional ML and encoder-based classifiers remain the better first choice.

References

- Anthropic. System card: Claude sonnet 4.6. Technical report, Anthropic, 2026. URL <https://www.anthropic.com/claude-sonnet-4-6-system-card>.
- Anna L. Buczak and Erhan Guven. A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Communications Surveys & Tutorials*, 18(2):1153–1176, 2016. doi: 10.1109/COMST.2015.2494502. URL <https://doi.org/10.1109/COMST.2015.2494502>.
- Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance, 2023. URL <https://arxiv.org/abs/2305.05176>.
- Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785. URL <https://doi.org/10.1145/2939672.2939785>.
- Aviad Cohen, Nir Nissim, and Yuval Elovici. Novel set of general descriptive features for enhanced detection of malicious emails using machine learning methods. *Expert Systems with Applications*, 110:143–169, 2018. doi: 10.1016/j.eswa.2018.05.031. URL <https://doi.org/10.1016/j.eswa.2018.05.031>.
- Jesse Davis and Mark Goadrich. The relationship between precision-recall and roc curves. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 233–240, 2006. doi: 10.1145/1143844.1143874. URL <https://doi.org/10.1145/1143844.1143874>.
- Nabila Farnaaz and M. A. Jabbar. Random forest modeling for network intrusion detection system. *Procedia Computer Science*, 89:213–217, 2016. doi: 10.1016/j.procs.2016.06.047. URL <https://doi.org/10.1016/j.procs.2016.06.047>.
- Gemma Team. Gemma 3 technical report, 2025. URL <https://arxiv.org/abs/2503.19786>.
- Google DeepMind. Gemini 3.1 pro. Technical report, Google, 2026. URL [\[TODO: addofficialGemini3.1Promodelcardlink\]](#).

- Stefan Hegselmann, Alejandro Buendia, Hunter Lang, Monica Agrawal, Xiaoyi Jiang, and David Sontag. TabLLM: Few-shot classification of tabular data with large language models. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 206, pages 5549–5581, 2023. URL <https://proceedings.mlr.press/v206/hegselmann23a.html>.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991. doi: 10.1162/neco.1991.3.1.79. URL <https://doi.org/10.1162/neco.1991.3.1.79>.
- Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. Mistral 7b, 2023. URL <https://arxiv.org/abs/2310.06825>.
- Takashi Koide, Naoki Fukushi, Hiroki Nakano, and Daiki Chiba. ChatSpamDetector: Leveraging large language models for effective phishing email detection, 2024. URL <https://arxiv.org/abs/2402.18093>.
- Zefang Liu. Phishing email dataset. Hugging Face Datasets, 2023. URL <https://huggingface.co/datasets/zefang-liu/phishing-email-dataset>.
- Llama Team, AI@Meta. The llama 3 herd of models, 2024. URL <https://ai.meta.com/research/publications/the-llama-3-herd-of-models/>.
- Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. Kitsune: An ensemble of autoencoders for online network intrusion detection. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2018. doi: 10.14722/ndss.2018.23204. URL <https://doi.org/10.14722/ndss.2018.23204>.
- Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data, 2024. URL <https://arxiv.org/abs/2406.18665>.
- OpenAI. gpt-oss-120b & gpt-oss-20b model card, 2025. URL <https://arxiv.org/abs/2508.10925>.
- Shahriyar Zaman Ridoy, Md. Shazzad Hossain Shaon, Alfredo Cuzzocrea, and Mst Shapna Akter. EnStack: An ensemble stacking framework of large language models for enhanced vulnerability detection in source code, 2024. URL <https://arxiv.org/abs/2411.16561>.
- Ozgur Koray Sahingoz, Ebubekir Buber, Onder Demir, and Banu Diri. Machine learning based phishing detection from urls. *Expert Systems with Applications*, 117:345–357, 2019. doi: 10.1016/j.eswa.2018.09.029. URL <https://doi.org/10.1016/j.eswa.2018.09.029>.
- Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116, 2018. doi: 10.5220/0006639801080116. URL <https://doi.org/10.5220/0006639801080116>.

Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, Nathan Cooper, Griffin Adams, Jeremy Howard, and Iacopo Poli. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference, 2024. URL <https://arxiv.org/abs/2412.13663>.

David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992. doi: 10.1016/S0893-6080(05)80023-1. URL [https://doi.org/10.1016/S0893-6080\(05\)80023-1](https://doi.org/10.1016/S0893-6080(05)80023-1).